

Industrial Training Report

Full-Stack Web Application Development & AI Integration Industrial Training at NEXONICA SYSTEMS PRIVATE LIMITED

Index

Sr. No	TITLE	Page. No
01	Organization Structure of Industry and General Layout	1-3
02	Introduction to Industry / Organization	4-5
03	Types of Major Equipment, Hardware, Software, Instruments and Raw Materials Used in Industry	6-7
04	Processes, Development Methodologies and Material Handling Procedures	8-9
05	Testing of Hardware, Software and Material Handling Procedures	10-11
06	Safety Procedures Followed and Safety Gears Used	12
07	Particulars of Practical Experiences in Industry / Organization	13-15
08	Detailed Report of the Tasks Undertaken During the Internship	16-18
09	Special / Challenging Experiences Encountered During the Internship	19
10	Assignments	20-21
11	Short report / Description of the project	22-24
12	Experience and working in industry photos	25-26
13	Conclusion	27
14	References	28-29

Chapter 1: Organization Structure of Industry and General Layout

1.1 Organization Structure of Nexonica Systems Private Limited

Nexonica Systems Private Limited is an information technology company headquartered in Nashik, Maharashtra. The company offers software development, website and mobile application engineering, IoT, AI/ML solutions and cloud computing services, and also runs dedicated industrial internship and training programs for engineering and diploma students. Because the company works across product development and training, interns get to see both the engineering side (real client-style projects) and the mentoring side (structured learning plans).

Particular	Details
Organization Name	Nexonica Systems Private Limited
Business Domain	Information Technology - Software Development & Technology Solutions
Registered Office	F-5, Ashtvinayak Park, Thatte Nagar, College Road, Nashik, Maharashtra - 422005
Official Website	www.nexonica.com
Core Focus Areas	Full-Stack Web Development, Mobile App Development, IoT, AI/ML, Cloud Computing, R&D and Student Internship Training
Technology Orientation	Java, React.js, Python, Flask, Android, Angular.js, Flutter, .NET, IoT, AI/ML, Data Science

Table 1.1 - Company profile

Major Business Verticals of Nexonica Systems: Each vertical is handled by specialists, but projects frequently combine more than one vertical - for example, a web portal with an AI-based recommendation module or an IoT dashboard backed by a cloud database.

Vertical	Description	Typical Technologies
Web Development	Responsive websites, enterprise portals, dashboards and single-page applications.	HTML5, CSS3, React.js, Angular.js, Flask, Java
Mobile App Development	Native and cross-platform applications for Android and other platforms.	Android, Flutter
IoT Solutions	Connecting sensors and devices to cloud platforms and dashboards for monitoring and control.	Embedded programming, MQTT/REST, cloud DB
AI / ML & Data Science	Data analysis, prediction models, and integration of intelligent features into applications.	Python, ML libraries, REST APIs
Cloud Computing	Deployment, hosting and scaling of applications on cloud infrastructure.	Cloud VMs, containers, managed databases
Training & Internship	Industrial training, mini-projects and capstone projects for students.	Full-stack curriculum, Git workflows

Table 1.2 - Business verticals of Nexonica Systems

1.2 Leadership

As per official company records, Nexonica Systems Private Limited is led by its Directors and Co-Founders, including Mr. Kanchan Prakash Ekhande. The leadership team oversees the organisation's technical direction, client engagement, and internship and training programs. They are also involved in selecting the technology stack for new projects and shaping the internship curriculum to ensure that it reflects the skills and technologies currently demanded by the industry.

1.3 Corporate Hierarchy & Teams

The organisation follows a structured hierarchy in which strategic decisions flow from the leadership team to the project management level and then to the respective execution teams. Interns are attached to the Training & Internship Cell and work closely with developers and testers under the guidance and supervision of a technical lead. This structure ensures effective coordination, proper task allocation, technical guidance, and smooth execution of projects.

Level	Team / Role	Responsibility
1	Directors / Founding Leadership	Provide strategic direction, oversee the overall technology vision and manage client relations.
2	Project Management & Technical Lead	Sprint planning, code reviews, architectural oversight and mentoring of interns.
3	Development Team	Full-Stack Engineers, Frontend Specialists (React/UI) and Backend Developers (Python/Flask/PostgreSQL).
3	Quality Assurance & Testing Team	API testing, functional validation and deployment pipeline integrity.
4	Training & Internship Cell	Coordinates onboarding, task allocation and academic documentation support for student interns.

Table 1.3 - Organisational layers and responsibilities

1.4 Training & Internship Cell

The Training & Internship Cell is the unit that interacts most closely with students and interns. Its responsibilities include onboarding interns, providing a structured learning roadmap, allocating mini-tasks of increasing difficulty, scheduling review sessions with mentors, and assisting students in preparing the documentation required by their respective institutes. The cell encourages interns to follow professional work practices similar to those followed by employees, including attending stand-

up meetings, committing code regularly, participating in code reviews, and responding to review comments.

1.5 General Layout and Work Culture

Interns work on developer laptops with stable broadband connectivity and use tools such as Visual Studio Code, web browsers with developer tools, PostgreSQL, pgAdmin, and Postman for development, database management, API testing, and debugging. The organisation follows an agile work routine consisting of stand-up meetings, sprint planning, reviews, and retrospectives. The following professional practices are observed:

Learning-Oriented Environment: Mistakes in code are treated as learning opportunities and are discussed constructively during reviews.

Documentation Habit: Each module is documented with its purpose, endpoints, functionality, and sample requests wherever required.

Transparency: Progress and challenges are discussed openly during stand-up meetings so that blockers can be identified and resolved at an early stage.

Confidentiality: Credentials, client information, project data, and source code are handled in accordance with company policies and security practices.

Punctuality and Ownership: Each intern is expected to take responsibility for assigned tasks from understanding the requirements through development, testing, documentation, and final submission.

Chapter 2: Introduction to Industry / Organization

2.1 Introduction

This report documents the industrial internship undertaken at Nexonica Systems Private Limited (www.nexonica.com), Nashik, an IT company offering services in IoT, AI/ML, full-stack development, cloud services, and structured internship programs for students. The internship was designed to bridge the gap between theoretical classroom learning in diploma engineering and practical, industry-standard software development methodologies. During the internship tenure, active participation was maintained across full-stack application modules, database schema design, RESTful API development, frontend interactivity using modern component libraries, and software testing. The training exposed the intern to real-world project workflows, professional tools, and collaborative development practices followed by the organisation's engineering teams.

2.2 Background

A diploma curriculum in Computer Engineering or Information Technology covers programming fundamentals, database management, web technologies, and software engineering. However, classroom exercises are usually small, single-developer programs with limited exposure to deployment, version control, and formal testing. Industry projects, on the other hand, involve multiple layers such as the user interface, server-side logic, and database, along with multiple developers, deadlines, and strict quality expectations. The internship was therefore planned to expose the student to this complete software development environment.

2.3 Objectives of the Internship

The major objectives of the internship were:

- To understand the structure and working culture of a software company.
- To learn the full-stack development workflow, from requirement analysis to documentation.
- To design responsive user interfaces using HTML5, CSS3, and React.js.
- To build REST APIs using Python and Flask and connect them to a PostgreSQL database.

- To practise software testing using unittest for unit testing and Postman for API testing.
- To use Git and GitHub for branching, pull requests, and conflict resolution.
- To develop a capstone project that integrates the technologies and concepts learned during the internship.

2.4 Scope of the Training

The scope of the training covered the three major tiers of a typical web application. In the presentation tier, the intern worked on static pages, responsive landing pages, and React components. In the application tier, the intern developed Flask routes, blueprints, validation logic, and error-handling mechanisms. In the data tier, the intern designed PostgreSQL database schemas, used pgAdmin 4 for database inspection and management, and used Flask-SQLAlchemy for ORM-based database queries. Cross-cutting concerns such as CORS configuration, environment variables, logging, and software testing were also included in the scope of the training.

2.5 Why Full-Stack Development?

A full-stack developer understands how data moves from a user's browser to the backend and database and then returns to the user interface. This end-to-end understanding makes it easier to debug problems, design efficient APIs, and communicate effectively with both frontend and backend development teams. Python with Flask was selected for backend development because of its simplicity, flexibility, and extensive library support. Its strong ecosystem also provides a suitable foundation for future AI and machine-learning integration.

Layer	Technology	Reason for Selection
Frontend	HTML5, CSS3, React.js	Component reuse, fast UI updates, and strong community support
Backend	Python, Flask	Lightweight framework, easy to learn, and extensive library support
ORM	Flask-SQLAlchemy	Maps Python classes to database tables and reduces the need for raw SQL
Database	PostgreSQL	ACID-compliant database supporting relationships, constraints, and JSON data
Tools	Postman, pgAdmin, Git	Industry-standard tools for API testing, database management, and version control

Table 2.1 – Technology Stack and Rationale

2.6 Expected Learning Outcomes

At the completion of the internship, the expected learning outcomes were:

- Ability to build a complete web application with separate frontend and backend components.
- Ability to design normalized database tables and write basic to intermediate SQL queries.
- Ability to document and test REST APIs using appropriate HTTP status codes and JSON responses.
- Familiarity with agile ceremonies, code reviews, and feature-branch workflows.
- Improved communication, time management, problem-solving, and teamwork skills.

2.7 Internship Details

Item	Details
Company	Nexonica Systems Private Limited, Nashik
Domain	Full-Stack Development (Python, Flask, React.js, PostgreSQL)
Duration	[1 June 2026] to [End Date]

Mode	Offline
Mentor / Guide	[Mentor Name]
Academic Year	2025–2026

Table 2.2 – Internship Particulars

Chapter 3: Types of Major Equipment, Hardware, Software, Instruments and Raw Materials Used in Industry

Professional software development at Nexonica Systems Private Limited requires a combination of capable developer hardware and an ecosystem of industry-standard software tools, code editors, database management systems, and development utilities. The following hardware and software tools were used during the internship.

3.1 Major Hardware and Equipment Used

Sr. No.	Equipment / Hardware	Purpose / Application
1	High-performance developer laptop (Intel Core i5/i7, 8–16 GB RAM, SSD storage)	Running local development servers, IDEs, databases, and development tools smoothly
2	Stable broadband internet connection	Version control using Git/GitHub, package installation, documentation access, and API testing
3	Dual / extended display (where available)	Parallel viewing of the code editor, browser, database tools, and API testing applications

Table 3.1 – Hardware Used

A solid-state drive (SSD) reduces the startup time of IDEs and databases, while 8 GB or more of RAM allows developers to keep Visual Studio Code, a browser with developer tools, PostgreSQL, pgAdmin, and Postman open simultaneously with minimal slowdown. A reliable internet connection is essential because package managers such as pip and npm download required libraries and dependencies. It is also required for pushing and pulling code from GitHub and accessing online development resources.

3.2 Major Software and Tools Used

Category	Tool	Purpose in Internship
IDE / Editor	Visual Studio Code	Primary code editing, debugging, extensions, and integrated terminal
Frontend	HTML5, CSS3, React.js	Building responsive user interfaces and single-page applications
Backend	Python, Flask, Flask-SQLAlchemy	Developing REST API routes, controllers, database operations, and business logic
Database	PostgreSQL, pgAdmin 4	Designing database schemas, managing tables, running SQL queries, and inspecting database records
API Testing	Postman	Testing REST endpoints such as GET, POST, PUT, and DELETE and validating JSON responses
Version Control	Git & GitHub	Tracking code changes, creating branches, and managing collaborative development and merging
Mobile / Preview	Android Studio / Device Preview	Testing responsive layouts and application interfaces across mobile screen sizes

Table 3.2 – Software and Tools Used

3.3 Tool-wise Description

Visual Studio Code: Visual Studio Code (VS Code) is a lightweight and extensible source-code editor used as the primary development environment. Its integrated terminal allows developers to run the

Flask backend server, React development server, Git commands, and other utilities without leaving the editor. Useful extensions include the Python extension for IntelliSense, linting, and debugging; ES7+ React snippets for faster React development; Prettier for code formatting; ESLint for identifying JavaScript and React coding issues; and GitLens for enhanced Git integration.

Python and Virtual Environments: Python was used for backend development during the internship. A separate virtual environment was created for each project to prevent dependency and library-version conflicts between projects. Project dependencies were recorded in a requirements.txt file. This allows other team members or developers to recreate the project environment with the required packages and versions.

Flask and Flask-SQLAlchemy: Flask is a lightweight Python web framework that provides routing, request and response handling, and a development server while allowing developers to select additional components according to project requirements. Flask-SQLAlchemy provides an Object-Relational Mapping (ORM) layer. Database tables can be represented as Python classes, and database operations can be performed using ORM methods. This simplifies database interaction and reduces the need to write repetitive raw SQL queries.

React.js: React.js is a JavaScript library used for building modern and interactive user interfaces using reusable components. During the internship, functional components, the useState hook for managing local state, and the useEffect hook for handling side effects such as API calls were used. Data was passed between components using props, allowing the application to maintain a structured and reusable component architecture.

PostgreSQL and pgAdmin 4: PostgreSQL is an open-source relational database management system known for reliability, scalability, and strong compliance with SQL standards. It was used to store and manage structured application data. pgAdmin 4 is a graphical administration and management tool for PostgreSQL. It was used to create databases and tables, view records, execute SQL queries through the Query Tool, and inspect relationships between database tables.

Postman, Git and GitHub: Postman is an API development and testing tool used to send HTTP requests to the Flask server and inspect response status codes, headers, and JSON data. API requests were organized into collections for systematic testing. Git is a distributed version-control system used to track source-code changes. GitHub provides remote repository hosting and collaboration features. During development, tasks were organized using separate branches, and completed work could be merged after review.

3.4 Raw Materials (Digital Inputs)

In a software development company, the concept of raw materials refers primarily to digital inputs and resources required for developing software applications. These include requirement documents, source code, database contents, third-party libraries and packages downloaded through pip and npm, configuration files, and authorized access credentials. These digital assets are important components of the development process and must be handled securely according to the organisation's policies. Sensitive information such as credentials, client data, and source code must be protected from unauthorized access.

3.5 Environment Setup Commands

The following commands summarize the basic environment setup performed during the initial stage of the internship (Task 01).

```
# 1. Create a Python virtual environment
python -m venv venv
# Activate the virtual environment on Windows
venv\Scripts\activate
# Activate the virtual environment on Linux / macOS
source venv/bin/activate
```

```
# 2. Install backend dependencies
pip install flask flask-sqlalchemy flask-cors psycopg2-binary python-dotenv
# Save installed dependencies
pip freeze > requirements.txt
# 3. Configure Git identity
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
# 4. Create a React application
npx create-react-app client
# Alternative: Create a React application using Vite
npm create vite@latest client
# Install Axios
cd client
npm install axios
```

Listing 3.1 – Environment Setup Commands

Chapter 4: Processes, Development Methodologies and Material Handling Procedures

Nexonica Systems follows structured and agile development workflows to ensure code quality, maintainability, and timely delivery of project modules.

4.1 Agile & Scrum Methodology

Work is divided into short iterations called sprints. Requirements are written as small user stories. For example: “As a user, I want to create a service request so that the support team can act on it.” Each story is estimated, assigned, developed, tested, and tracked until completion.

Scrum Ceremony	Purpose	Intern's Participation
Daily Stand-up	Short meeting to discuss completed work, planned work, and blockers	Reported task status daily
Sprint Planning	Selecting and planning stories for the upcoming sprint	Received and clarified assigned tasks
Sprint Review / Demo	Demonstrating completed work to mentors and stakeholders	Demonstrated working APIs and UI screens
Retrospective	Discussing what went well and what can be improved	Shared difficulties, observations, and learnings

Table 4.1 – Scrum Ceremonies

4.2 Standard Development Cycle

The standard development cycle followed during the internship was: Requirement Analysis → Planning → Architecture & Database Design → Implementation → API Testing → Debugging → Documentation.

Stage	Activities Performed
Requirement Analysis	Reading the task description, identifying inputs and outputs, and clarifying doubts with the mentor
Planning	Breaking the task into smaller steps, estimating effort, and creating a feature branch
Architecture & Database Design	Deciding tables, columns, keys, and relationships and defining API endpoints
Implementation	Writing Flask routes and models, building React components, and connecting them to APIs
API Testing	Sending requests using Postman and verifying status codes and response payloads
Debugging	Reading stack traces and logs and fixing validation, application, and database errors
Documentation	Updating the README, API endpoint tables, and internship documentation

Table 4.2 – Activities in Each Development Stage

4.3 Feature-Branch Git Workflow

The main branch contains stable and reviewed code. For each task, a new branch is created from the main branch. The developer works on the task, commits changes in small increments, and raises a pull request when the task is ready. After review and approval, the branch is merged into the main branch and can be deleted.

```
git checkout main
git pull origin main
# Create a task branch
git checkout -b feature/flask-crud
git add .
git commit -m "feat: add create and list endpoints for items"
git push origin feature/flask-crud
# After review and merge
git checkout main
git pull origin main
git branch -d feature/flask-crud
```

Listing 4.1 – Typical Git Workflow for One Task

Prefix	Meaning	Example Commit Message
feat	A new feature	feat: add status filter to request list
fix	A bug fix	fix: return 404 for unknown request id
docs	Documentation changes only	docs: add API endpoint table to README
test	Adding or fixing tests	test: add unit tests for item model
refactor	Code restructuring without behaviour change	refactor: move routes into blueprint

Table 4.3 – Commit Message Convention

4.4 Code Review & Coding Standards

Peer code review is performed before merging changes into the main branch. The following coding and quality standards are followed:

Meaningful Naming: Names should be clear and follow PEP 8 conventions for Python and camelCase/PascalCase conventions for JavaScript and React.

Credential Security: Credentials, passwords, and API keys must not be hard-coded. Environment variables are used instead.

Input Validation: User inputs are validated, and appropriate HTTP status codes are returned for errors.

Database Safety: Database operations are handled using appropriate exception handling and rollback mechanisms when failures occur.

Code Reusability: Functions should be short and focused on a single purpose, and unnecessary code duplication should be avoided.

Documentation: Comments and docstrings should explain the purpose or reasoning behind the code where necessary.

Python Standards: PEP 8 style, snake_case naming, docstrings, and four-space indentation are followed.

JavaScript / React Standards: Functional components, ES6 syntax, const/let, and organized component files are preferred.

SQL Standards: Lowercase snake_case naming and explicit primary and foreign keys are used.

API Design: Plural nouns are used in URLs, HTTP methods represent operations, and JSON responses follow a consistent structure.

4.5 Handling Procedure for Each Request

Every API request handled by the Flask backend passes through a standard sequence of steps. Following a fixed request-handling pipeline makes system behaviour predictable and simplifies debugging and maintenance.

Step	Description	Example
1. Reception	Flask matches the requested URL and HTTP method with the appropriate route function	POST /api/items
2. Validation	Mandatory fields, data types, and input lengths are checked	name must not be empty
3. Business Rules	Application-specific logic and default values are applied	Default status = Open
4. Database Transaction	Data is added or modified and changes are committed using SQLAlchemy	db.session.add(item) → commit()
5. Exception Handling	Errors are caught, transactions are rolled back, and appropriate error responses are generated	400 / 404 / 500 with an error message
6. Response	A JSON response is returned with the appropriate HTTP status code	{"id": 5} with 201 Created

Table 4.4 – Request Handling Pipeline

Chapter 5: Testing of Hardware, Software and Material Handling Procedures

Software reliability was ensured through structured testing covering unit-level logic, cross-layer integration, and complete API validation. Testing was treated as an integral part of development rather than a separate activity performed at the end. Every API route developed was tested using Postman and then covered by automated tests wherever applicable.

5.1 Levels of Testing

Level	What Is Tested	Tool Used
Unit Testing	Individual Python functions, model methods, and Flask route controllers tested independently	Python unittest
Integration Testing	Communication between the React frontend, Flask backend, and PostgreSQL database	Browser DevTools, Postman
API End-to-End Testing	HTTP GET, POST, PUT, and DELETE requests, including status codes and JSON payloads	Postman
UI / Responsive Testing	Layout behaviour across desktop and mobile viewports	Browser Device Mode, Android Studio Preview

Table 5.1 – Testing Levels

5.2 Integration Testing

Integration testing verifies that the different layers of the application work together correctly. A typical test involved filling the New Request form in React, submitting the form, confirming that Axios sends a POST request, verifying that Flask returns an appropriate response such as 201 Created, and finally checking PostgreSQL through pgAdmin to confirm that the new record exists. Common issues identified during integration testing included incorrect CORS configuration, mismatched JSON field names, and data-type mismatches between the frontend and backend.

5.3 API End-to-End Testing with Postman

Postman was used to validate the API endpoints. For each request, the HTTP status code, response headers such as Content-Type: application/json, JSON response structure, and response time were checked. Postman's Tests tab was used to create JavaScript-based assertions. This allowed API collections to be executed repeatedly and verified automatically.

```
// Postman "Tests" tab for POST /api/items
pm.test("Status code is 201", function () {
    pm.response.to.have.status(201);
});
pm.test("Response has id", function () {
    var json = pm.response.json();
    pm.expect(json).to.have.property("id");
    pm.environment.set("item_id", json.id);
});
```

Listing 5.2 – Postman Assertions

5.4 Functional Test Cases

Test ID	Test Case	Expected Result	Result
TC-01	Create a new record with valid parameters	Record stored – HTTP 201 Created	Pass
TC-02	Retrieve all records	JSON list returned – HTTP 200 OK	Pass
TC-03	Retrieve a specific record using a valid ID	Correct record details returned	Pass
TC-04	Update an existing record	Record updated in the database	Pass
TC-05	Delete a specific record	Record removed from the database	Pass
TC-06	Submit a request with missing mandatory fields	Validation error – HTTP 400	Pass
TC-07	Query using a non-existent record ID	Controlled “Not Found” response – HTTP 404	Pass
TC-08	Create a record with an incorrect data type	Validation error – HTTP 400	Pass
TC-09	Update a record that does not exist	Controlled “Not Found” response – HTTP 404	Pass
TC-10	Delete a record that has already been deleted	Controlled “Not Found” response – HTTP 404	Pass
TC-11	Call the API from the React frontend using a different origin	Request succeeds because CORS is enabled	Pass
TC-12	Filter requests by status “Open”	Only records with status “Open” are returned	Pass
TC-13	Open the dashboard on a mobile viewport	Layout adapts correctly without horizontal scrolling	Pass

Table 5.2 – Functional Test Cases

5.6 Defect Handling

When a test fails, the defect is recorded along with the steps to reproduce it, the expected result, and the actual result. The developer then reproduces the issue, fixes it on the same feature branch, adds or updates a test to prevent the issue from recurring, and re-runs the complete test suite before requesting another review.

Defect Handling Flow: Detect → Report → Reproduce → Fix → Re-test → Close

5.7 Digital Asset & Material Handling

In a software development company, the most sensitive materials include source code, database contents, configuration files, and access credentials. These digital assets must be handled securely and according to organisational policies. The detailed procedures for handling sensitive digital assets, credentials, and project materials are described in Chapter 6.

Chapter 6: Safety Procedures Followed and Safety Gears Used

For a software organisation, safety primarily involves protecting employee work, client data, source code, credentials, and system availability. Nexonica Systems follows defined safety and security procedures, and interns are expected to follow these practices from the beginning of their internship.

6.1 Professional Workplace Practices

The following professional safety practices were followed during the internship:

Confidentiality: Credentials, client data, and source code were handled according to company policies.

Clean Workstation: Passwords and confidential information were not included in screenshots, chats, or shared documents.

Access Control: Repository and database access was provided only to authorised personnel who required it.

Transparency: Technical blockers, security concerns, and project risks were communicated at an early stage during discussions and daily updates.

6.2 Cyber Security and Information Safety

The following practices were used to protect applications, databases, and development environments:

Environment Variables: Database URLs, secret keys, and passwords were stored in a .env file. The file was added to .gitignore and was not committed to GitHub.

Password Security: User passwords were stored as salted hashes rather than plain-text passwords.

SQL Injection Protection: SQL queries were not manually constructed using untrusted user input. ORM-based database operations were used to reduce SQL injection risks.

Input Validation and Controlled CORS: User input was validated before database operations, and Cross-Origin Resource Sharing (CORS) was configured according to application requirements.

Token-Based Authentication: Protected routes verified authentication tokens before processing authorised requests.

Code Review: Code was reviewed before merging. Hard-coded credentials were avoided, errors were handled properly, and database operations used exception handling and rollback mechanisms.

```
# .env (never committed)
DATABASE_URL=postgres://postgres:password@localhost:5432/service_portal
SECRET_KEY=change-this-in-production
```

Listing 6.1 – Environment File (Placeholder Values)

Note: The values shown above are placeholders and should not contain real production credentials.

6.3 Data Backup and Recovery

Regular database backups can be created using PostgreSQL's pg_dump utility and stored in a protected location. Source code was maintained and backed up through Git and GitHub. Feature branches were used for individual tasks until the changes were reviewed and merged.

```
pg_dump -U postgres -d service_portal -F c -f backup_service_portal.dump
pg_restore -U postgres -d service_portal_restore backup_service_portal.dump
```

Listing 6.2 – Backup and Restore Commands

These procedures help reduce the risk of data loss and provide a method for restoring database information when required.

6.4 Safety Gears (Digital Safeguards) Used

In a software development environment, safety gears are mainly implemented as digital security controls and development practices.

Safety Gear / Safeguard	Protection Provided
.env file + .gitignore	Prevents accidental exposure of passwords and secret keys
Password hashing (Werkzeug)	Protects passwords from being stored in readable form
ORM / Parameterised Queries	Reduces the risk of SQL injection

Git Feature Branches + Pull Requests	Prevents unreviewed or unstable code from directly reaching the main branch
pg_dump Backups	Protects against database data loss
Separate Test Database / In-Memory SQLite	Reduces the risk of tests affecting production data
Role-Based Access	Restricts users from accessing features that are not permitted for their role

Table 6.1 – Digital Safeguards Used

The above safety procedures and digital safeguards helped maintain confidentiality, integrity, availability, and controlled access throughout the software development process.

Chapter 7: Particulars of Practical Experiences in Industry / Organization

The internship provided progressive hands-on experience, beginning with fundamental UI development and progressing to backend development, database integration, API testing, and complete application workflows. The major technologies and practices learned during the internship include:

- HTML5, CSS3 and Responsive Design – semantic structure, styling, Flexbox, CSS Grid and media queries.
- React.js – functional components, props, useState and useEffect.
- Python and Flask – route handling, blueprints and application structuring.
- PostgreSQL and Flask-SQLAlchemy – database schema design, relationships and ORM queries.
- REST APIs – API design and frontend consumption using Axios/fetch.
- Postman – endpoint testing, environment variables and collections.
- pgAdmin 4 – table design, query execution and database inspection.
- Git and GitHub – branching, commits, pull requests and merge conflict resolution.

7.1 Frontend Experience: HTML5, CSS3 and Responsive Design

The first phase of the internship focused on developing static web pages using semantic HTML5 elements such as <header>, <nav>, <main>, <section>, and <footer>. CSS3 was then used to style the pages and create responsive layouts. Flexbox and CSS Grid were used to arrange page elements, while media queries were applied to adapt layouts for smaller screens.

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 16px;
}
@media (max-width: 600px) {
  .navbar {
    flex-direction: column;
  }
}
```

Listing 7.1 – Responsive Card Grid Using CSS Grid

This practical work helped in understanding how a website can maintain usability and visual consistency across different screen sizes.

7.2 React.js Experience

React.js introduced a component-based approach to frontend development. The interface was divided into reusable components such as Navbar, RequestCard, RequestForm, and StatusBadge. Data was passed between components using props. An important learning area was handling asynchronous data. The application displayed a loading state while waiting for the API response and then displayed either the received data or an error message.

7.3 Backend Experience: Python and Flask

On the backend, practical experience was gained in understanding how Flask maps URLs to Python functions, reads JSON request bodies using `request.get_json()`, and returns JSON responses using `jsonify()`. As the number of routes increased, Flask Blueprints were used to organise related routes into separate modules. The Application Factory Pattern was also introduced to create the Flask application with different configurations for development and testing. This approach improved understanding of backend application structure, modularity, and maintainability.

7.4 Database Experience: PostgreSQL and SQLAlchemy

Working with PostgreSQL provided practical knowledge of database design and data integrity. Primary keys, foreign keys, unique constraints, and NOT NULL constraints were used to maintain consistent data. Relationships such as one category having many requests were modelled using foreign keys and SQLAlchemy's `relationship()` functionality. SQL queries were first practised using the pgAdmin Query Tool and were then implemented as ORM queries using SQLAlchemy.

```
SELECT
    r.id,
    r.title,
    c.name AS category,
    r.status
FROM requests r
JOIN categories c ON c.id = r.category_id
WHERE r.status = 'Open'
ORDER BY r.created_at DESC;
```

Listing 7.3 – Example JOIN Query Run in pgAdmin 4

This practical work helped in understanding how related records can be retrieved and filtered from multiple database tables.

7.5 Integration Experience: CORS and Axios

During frontend-backend integration, the React application running on one port attempted to communicate with the Flask API running on another port. The browser initially blocked the request because of the same-origin policy. The issue was resolved by configuring Flask-CORS on the backend and sending the appropriate headers. This provided practical understanding of browser security, Cross-Origin Resource Sharing, and frontend-backend communication.

7.6 Flask & Database CRUD Operations

A core competency developed during the internship was implementing complete CRUD (Create, Read, Update, Delete) operations using Python Flask and PostgreSQL. These four operations form the foundation of many business applications.

Operation	HTTP Method	Purpose	Example Route	SQL
Create	POST	Insert a new record	/api/items	INSERT
Read	GET	Fetch one or multiple records	/api/items or /api/items/{id}	SELECT
Update	PUT / PATCH	Modify an existing record	/api/items/{id}	UPDATE
Delete	DELETE	Remove a record	/api/items/{id}	DELETE

Table 7.1 – CRUD Operations Mapped to HTTP Methods and SQL

A recommended Flask project structure used for organising the application is shown below.

```
app.py          # Application factory, CORS, blueprint registration
config.py       # Reads settings from environment variables
models.py       # SQLAlchemy models
routes/
├── __init__.py
└── items.py     # Blueprint with CRUD routes
```

```
tests/
└─ test_items.py
requirements.txt
.env
```

Listing 7.4 – Recommended Flask Project Structure

The database model was created using Flask-SQLAlchemy.

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime
db = SQLAlchemy()

class ItemModel(db.Model):
    __tablename__ = "items"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    description = db.Column(db.Text)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def to_dict(self):
        return {
            "id": self.id,
            "name": self.name,
            "description": self.description,
            "created_at": self.created_at.isoformat()
        }
```

Listing 7.5 – SQLAlchemy ORM Model

The ORM model demonstrated how Python classes can represent database tables and how SQLAlchemy can be used to perform database operations without writing every SQL statement manually.

7.7 Challenges Faced and Solutions

During practical development, several technical challenges were encountered. These issues helped develop debugging, problem-solving, and integration skills.

Challenge	Cause	Solution
CORS error in browser console	Frontend and backend were running on different ports	Enabled Flask-CORS for the API
psycopg2 installation failed	Missing build dependencies	Used the psycopg2-binary package
400 error on POST from React	JSON field names differed from backend fields	Aligned field names and verified the request in Postman
Merge conflict in shared file	Two branches modified the same lines	Resolved the conflict manually, re-tested, and committed the merge
Data not updating on screen	React state was not updated after the API call	Updated the state or re-fetched the list after a successful request
Table not found error	Database tables had not been created	Created tables using db.create_all() and verified them in pgAdmin

Table 7.2 – Challenges Faced and Solutions

These challenges provided practical experience in identifying errors, analysing their causes, applying appropriate solutions, and verifying the results after fixing them.

Chapter 8: Detailed Report of the Tasks Undertaken During the Internship

Throughout the internship, multiple structured tasks and mini-projects were completed under mentor guidance. These included daily programming exercises, technology-specific activities, database and API development, frontend-backend integration, testing, version control, and documentation.

Task ID	Task Name & Description	Tools / Technology Used	Status
Task 01	Company onboarding; setup of VS Code, Python virtual environment, and Git	VS Code, Git	Completed
Task 02	Designed static HTML/CSS pages and practised responsive grid layouts	HTML5, CSS3	Completed
Task 03	Built small React UI components such as buttons, cards, and forms	React.js	Completed
Task 04	Designed a responsive landing page with modern styling and typography	HTML5, CSS3	Completed
Task 05	Developed interactive SPA features using React hooks and props	React.js, JavaScript	Completed
Task 06	Configured PostgreSQL schemas and managed relational tables using pgAdmin 4	PostgreSQL, pgAdmin 4	Completed
Task 07	Built RESTful Flask API endpoints supporting CRUD operations	Python, Flask, SQLAlchemy	Completed
Task 08	Wrote unit tests for Flask routes and model functions	Python, unittest	Completed
Task 09	Conducted API testing and validated headers, payloads, and error handling	Postman	Completed
Task 10	Integrated the React frontend with Flask APIs using CORS and Axios	React.js, Flask, Axios	Completed
Task 11	Practised Git branching, commits, pull requests, and merge conflict resolution	Git, GitHub	Completed
Task 12	Prepared technical documentation and the final capstone project report	MS Word, Markdown	Completed

Table 8.1 – Summary of Tasks

8.1 Task 01 – Onboarding and Environment Setup

Objective: To prepare a working development environment and understand the organisation's development processes.

Activities: Attended the orientation session, installed VS Code with the required extensions, created a Python virtual environment, installed and configured Git, configured the Git user identity, and cloned the initial practice repository from GitHub.

Outcome: A functional development workstation was prepared, along with an understanding of stand-up timings, task reporting procedures, and basic coding standards.

8.2 Task 02 – Static HTML/CSS Pages

Objective: To revise web development fundamentals and practise responsive layouts.

Activities: Created multi-section web pages using semantic HTML5 elements and styled them using external CSS. Flexbox, CSS Grid, and media queries were used to make layouts adaptable to mobile phones, tablets, and desktop screens.

Outcome: Developed clean and responsive static web pages that were reviewed by the mentor and later used as a foundation for subsequent tasks.

8.3 Task 03 – React UI Warm-up Components

Objective: To understand component-based frontend development.

Activities: Built reusable Button, Card, and Form components. Practised passing data through props, handling input fields and form submission events, using JSX syntax, and implementing conditional rendering.

Outcome: Created a small collection of reusable React components that could be integrated into later frontend tasks.

8.4 Task 04 – Responsive Landing Page

Objective: To apply modern web design principles to a complete webpage.
Activities: Designed a landing page containing a hero section, feature cards, call-to-action section, and footer. Selected suitable colours and typography, maintained readable contrast, and tested the design across different screen widths.
Outcome: Developed a polished and mobile-friendly landing page with a consistent visual design.

8.5 Task 05 – Interactive SPA with React Hooks

Objective: To develop interactive features using React state and side effects.
Activities: Used useState for form fields, counters, and filters and useEffect for loading data when components were mounted. Search and filter controls were implemented along with a modal form for adding items.
Outcome: Developed a single-page application with dynamic behaviour, interactive controls, and data updates without requiring full-page reloads.

8.6 Task 06 – PostgreSQL Schema Design in pgAdmin 4

Objective: To design relational database tables for the application.
Activities: Created the database and defined tables using primary keys, foreign keys, NOT NULL, and UNIQUE constraints. Sample records were inserted, and SQL queries using JOIN, WHERE, ORDER BY, and GROUP BY were practised in the pgAdmin Query Tool.
Outcome: Developed a structured and normalised database schema for users, categories, requests, and status history.

8.7 Task 07 – Flask REST API with CRUD

Objective: To develop the backend service for the application.
Activities: Created database models using Flask-SQLAlchemy, organised routes using Flask Blueprints, and implemented POST, GET, PUT, and DELETE endpoints. Input validation and consistent JSON error responses were also implemented.
Outcome: Developed functional REST API endpoints connected to the PostgreSQL database.

8.8 Task 08 – Unit Testing

Objective: To verify application functionality through automated testing.
Activities: Created unittest test cases using the Flask test client for both successful and failure scenarios. A temporary test database was used, and tests were executed before submitting changes for review.
Outcome: Developed a repeatable test suite that helped identify errors and prevent regressions during development.

8.9 Task 09 – API Testing with Postman

Objective: To verify API behaviour and endpoint functionality.
Activities: Created a Postman collection with environment variables and tested CRUD requests. Response headers, JSON request and response bodies, status codes, and error cases were validated. Assertions were also written using the Postman Tests tab.
Outcome: Prepared a reusable Postman collection that provided structured testing and documentation for the API endpoints.

8.10 Task 10 – Frontend-Backend Integration

Objective: To connect the React frontend with the Flask backend API.
Activities: Configured CORS in Flask and used Axios for GET, POST, PUT, and DELETE requests. Loading and error states were managed in React, and the UI state was refreshed after successful

operations.

Outcome: Established a complete full-stack workflow in which user actions were processed by the React interface, sent to the Flask API, stored in PostgreSQL, and reflected back on the screen.

8.11 Task 11 – Git Branching and Merge Conflicts

Objective: To understand safe collaboration and version control using Git.

Activities: Created feature branches, made small and meaningful commits, opened pull requests, responded to review comments, and resolved merge conflicts by analysing conflict markers and re-testing the affected code.

Outcome: Developed practical confidence in team-based version control, branch management, code review, and conflict resolution.

8.12 Task 12 – Documentation and Final Report

Objective: To document the technical work in a professional and structured manner.

Activities: Prepared README files, API endpoint tables, relevant screenshots, project documentation, and the final internship report according to the format required by the institute.

Outcome: Completed the technical documentation for the project and prepared the final internship report for academic submission.

Chapter 9: Special / Challenging Experiences Encountered During the Internship

9.1 Moving from Classroom Programs to Layered Applications

One of the initial challenges was understanding how a real application differs from classroom exercises. A real application has a user interface, server-side logic, and a database that must work together, along with version control and formal testing.

9.2 Technical Challenges and How They Were Solved

Challenge	Cause	Solution
CORS error in browser console	Frontend (port 3000) and backend (port 5000) ran on different origins	Enabled Flask-CORS for the API
psycopg2 installation failed	Missing build dependencies	Installed psycopg2-binary package
400 error on POST from React	JSON keys differed from backend fields	Aligned field names and checked the request in Postman first
Merge conflict in shared file	Two branches edited the same lines	Resolved manually, re-tested, and committed the merge
Data not updating on screen	State was not updated after the API call	Refreshed state or re-fetched the list after success
Table not found error	Migrations / create_all had not been run	Created tables using db.create_all() and verified them in pgAdmin

Table 9.1 – Challenges and Solutions

9.3 Asynchronous State in React

Handling asynchronous state by showing a Loading message, followed by the received data or an error message, was an important learning experience. Refreshing the screen correctly after every successful create, update, or delete operation was also practised.

9.4 Frontend-Backend Integration

Incorrect JSON field names, data-type mismatches, and CORS settings were common causes of integration failures. Testing each endpoint in Postman first and then connecting it to React made these problems easier to isolate and resolve.

9.5 Team Workflow

Working with feature branches, pull requests, and code review required adapting to structured team practices. Feedback was incorporated carefully, and merge conflicts were resolved and tested before completing the merge. Small, frequent commits and short-lived feature branches made collaboration easier and reduced the risk of integration problems.

9.6 Time Management

Tasks had to be completed within sprint timelines while still allowing sufficient time for debugging, testing, and documentation. Proper task prioritisation helped maintain steady progress throughout the internship.

9.7 Soft Skills Developed

The internship helped develop the following professional skills:

Communication: Explaining progress and problems clearly during stand-ups.

Time Management: Completing tasks within planned timelines.

Teamwork: Working with reviewers and accepting feedback positively.

Problem Solving: Reading error messages, referring to documentation, and testing possible solutions.

Professional Discipline: Maintaining regular commits, documentation, and testing habits.

Chapter 10: Assignments

10.1 Assignments

The practical assignments completed during the internship are summarised below along with their key code and implementation details. Screenshots of the outputs are provided in Chapter 12.

Assignment 1 – Environment Setup and Git Configuration: A Python virtual environment was created, Flask and related packages were installed, Git was configured, and a React application was created. The environment setup commands are provided in Listing 3.1.

Assignment 2 – Responsive Layout with CSS Grid: A responsive card grid was developed using CSS Grid. The layout automatically rearranged itself according to the available screen width. The implementation is provided in Listing 7.1.

Assignment 3 – React Component for Fetching API Data: A RequestList component was developed using useState, useEffect, and Axios to retrieve data from the Flask API and display it in the React interface. The implementation is provided in Listing 7.2.

Assignment 4 – SQL JOIN Query in pgAdmin 4: A SQL JOIN query was written to connect the requests and categories tables. The records were filtered according to status and sorted by creation date. The query is provided in Listing 7.3.

Assignment 5 – Flask CRUD Routes: Flask REST API routes were implemented for GET, POST, PUT, and DELETE operations. Input validation, database rollback on failure, and consistent JSON error responses were included.

```
@api_blueprint.route('/items/<int:item_id>', methods=['GET'])
def get_item(item_id):
    item = db.session.get(ItemModel, item_id)
    if not item:
```

```

        return jsonify({"error": "Item not found"}), 404
    return jsonify(item.to_dict()), 200

@api_blueprint.route('/items', methods=['POST'])
def create_item():
    data = request.get_json(silent=True) or {}
    if not data.get("name"):
        return jsonify({"error": "'name' is required"}), 400
    item = ItemModel(
        name=data["name"],
        description=data.get("description")
    )
    try:
        db.session.add(item)
        db.session.commit()
    except Exception:
        db.session.rollback()
        return jsonify({"error": "Database error"}), 500
    return jsonify(item.to_dict()), 201

@api_blueprint.route('/items/<int:item_id>', methods=['PUT'])
def update_item(item_id):
    item = db.session.get(ItemModel, item_id)
    if not item:
        return jsonify({"error": "Item not found"}), 404
    data = request.get_json(silent=True) or {}
    item.name = data.get("name", item.name)
    item.description = data.get("description", item.description)
    db.session.commit()
    return jsonify(item.to_dict()), 200

@api_blueprint.route('/items/<int:item_id>', methods=['DELETE'])
def delete_item(item_id):
    item = db.session.get(ItemModel, item_id)
    if not item:
        return jsonify({"error": "Item not found"}), 404
    db.session.delete(item)
    db.session.commit()
    return jsonify({"message": "Item deleted"}), 200

```

Listing 10.1 – Get, Create, Update, and Delete API Routes

Operation	Request	Response
Create	POST /api/items { "name": "Laptop", "description": "Not booting" }	201 Created { "id": 1, "name": "Laptop", ... }
Read All	GET /api/items	200 OK [{ "id": 1, ... }, { "id": 2, ... }]
Read One	GET /api/items/1	200 OK { "id": 1, "name": "Laptop", ... }
Update	PUT /api/items/1 { "name": "Laptop - repaired" }	200 OK Updated record
Delete	DELETE /api/items/1	200 OK { "message": "Item deleted" }
Invalid POST	POST /api/items { }	400 Bad Request { "error": "'name' is required" }

Table 10.1 – Sample API Requests and Responses

Assignment 6 – Unit Testing with unittest: Unit tests were written for both successful and failure scenarios using the Flask test client. The test implementation is provided in Listing 5.1.

Assignment 7 – API Testing with Postman: A Postman collection was created with environment variables and assertions. CRUD endpoints, response status codes, headers, JSON payloads, and error cases were tested. The testing script is provided in Listing 5.2.

Assignment 8 – Password Hashing: Password hashing was implemented using Werkzeug security functions. Passwords were stored as hashes rather than plain text.

```
from werkzeug.security import generate_password_hash, check_password_hash
# At registration
user.password_hash = generate_password_hash(data["password"])
# At login
ok = check_password_hash(
    user.password_hash,
    data["password"]
)
```

Listing 10.2 – Password Hashing

Assignment 9 – Git Feature-Branch Workflow: A feature branch was created for development, changes were committed using the feat, fix, and docs commit conventions, and a pull request was raised for review. A merge conflict was also resolved and the affected code was re-tested. The workflow is provided in Listing 4.1.

Assignment 10 – SQL Statements for CRUD: SQL statements were written to perform database table creation and CRUD operations.

```
CREATE TABLE items (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
-- Create
INSERT INTO items (name, description)
VALUES ('Laptop', 'Not booting');
-- Read all
SELECT * FROM items;
-- Read one
SELECT * FROM items
WHERE id = 1;
-- Update
UPDATE items
SET name = 'Laptop - repaired'
WHERE id = 1;
-- Delete
DELETE FROM items
WHERE id = 1;
```

Listing 10.3 – SQL Statements for CRUD Operations

These assignments provided practical experience in frontend development, backend API development, database operations, testing, security, version control, and full-stack application integration.

Chapter 11: Short Report / Description of the Project

11.1 Project Title

Project Name: LibraAI – AI-Based Institutional Library Management System

Languages / Technologies Used:

1. Frontend Tier: React 19, TypeScript 5.7, Vite 6.2, Tailwind CSS 3.4, Lucide React icons, Axios HTTP client, HTML5-QRCode optical scanner library.
2. Backend Tier: Node.js runtime environment, Express 4.21 web framework, TypeScript 5.7, Prisma ORM 6.4,
3. Database Infrastructure: PostgreSQL 15 relational database hosted on Supabase Cloud,
4. AI & Intelligence Services: OpenRouter API integrating Google Gemini 2.5 Flash

11.2 Project Overview

- **Objective:** To modernize and centralize higher education library operations by replacing error-prone manual ledgers and isolated spreadsheets with an automated, role-based cloud platform integrated with conversational AI study assistants and real-time business intelligence analytics.
- **System Architecture:** A decoupled three-tier client-server architecture. The presentation layer (React SPA) dispatches authenticated HTTPS/JSON requests with Bearer JWT headers to the Express application server, which executes business logic and interfaces with a Supabase-managed PostgreSQL database via Prisma ORM.
- **Key Features:** Granular 4-tier Role-Based Access Control (Student, Faculty, Librarian, Administrator); an institutional catalog holding 2,299 normalized titles and 8,246 physical copies imported via Excel; barcode-driven rapid circulation (issue, return, renewal, reservation); automated overdue fine calculation (₹5/day); interactive 22-chart Recharts analytics dashboard; LibraBot LLM study assistant; syllabus-to-book matcher; real-time reading hall seat reservation; and cryptographic SHA-256 activity audit logging.
- **Functional Modules:** Guest & Student Discovery Portal, Faculty Curriculum Shelf, Librarian Circulation Desk, Administrative Control Hub, AI Semantic Services & Study Assistant, and Executive Analytics & Reporting Module.

11.3 Problem Statement

Academic institutions frequently struggle with outdated, manual library administration systems or fragmented spreadsheet trackers. In such legacy environments, locating physical textbooks across vast campus collections requires laborious physical searches, while manual record-keeping leads to unrecorded loans, misplaced inventory, and overlooked overdue fines. Library administrators lack real-time visibility into collection turnover, seat occupancy, and department-wise demand trends. Furthermore, students receive no intelligent study support, personalized reading recommendations, or rapid syllabus alignment. A centralized, cloud-hosted, and AI-enabled library management system is required to eliminate operational inefficiencies, enforce strict audit trails, and elevate academic research capabilities.

11.4 Existing System vs Proposed System

Aspect	Existing (Manual / Spreadsheet) Approach	Proposed System (LibraAI)
Catalog Discovery	Paper index cards or static Excel files; slow manual keyword lookups.	Instant multi-attribute & semantic vector search across 8,246 physical copies.
Circulation Processing	Handwritten ledger entries prone to human transcription errors.	Barcode-driven rapid circulation with automated status and inventory validation.
Due Dates & Fines	Manual date calculation; overdue returns frequently missed.	Automated 14-day tracking with programmatic fine calculation (₹5/day).
Reports & Analytics	Tedious end-of-semester tallying; no graphical trend charts.	Live 22-chart Recharts dashboard showing footfall and category distributions.
Security & RBAC	Shared spreadsheets with zero access restrictions or audit logs.	Secure JWT tokens, bcrypt password hashing, and 4-tier granular RBAC.
Accessibility	Restricted strictly to on-premise physical library hours.	24/7 web access across desktop and mobile devices via cloud PostgreSQL.
Student Assistance	Dependent on staff availability; no study guidance tools.	24/7 LibraBot AI chatbot, syllabus matcher, and PDF study summaries.
Facility Management	Unmonitored reading halls; no seat reservation records.	Interactive digital seat booking with live zone occupancy tracking.

Table 11.1 - Comparison of systems

11.5 System Architecture

LibraAI is built upon a layered three-tier architectural paradigm designed for scalability, security, and high maintainability. The client-side Single Page Application (SPA) dispatches asynchronous HTTPS requests carrying JSON payloads and Bearer JWT authorization tokens to the Express application tier. The backend router layer enforces strict authentication and role-based authorization filters before delegating requests to controller services. Data access operations are executed through Prisma ORM, which generates type-safe SQL queries against the cloud PostgreSQL database. For intelligent services, the backend dispatches contextual prompts to OpenRouter LLM endpoints, utilizing SHA-256 database caching to minimize latency and API overhead.

Architecture Layer	Primary Responsibilities	Technologies Used
Presentation Layer	User interfaces, responsive Recharts visualization, barcode scanning, client routing.	React 19, TypeScript, Vite 6, Tailwind CSS, Axios
Application Tier	RESTful routing, JWT authentication, RBAC middleware, business logic, fine engine.	Node.js, Express 4.21, TypeScript, Zod, bcryptjs
Data Access Layer	Type-safe database abstraction, connection pooling, schema migrations.	Prisma ORM 6.4 Client & Query Engine
Data Storage Tier	Relational persistence, ACID transactions, foreign keys, B-tree indexes.	PostgreSQL 15 (Supabase Cloud Database)
AI & Intelligence	Conversational Q&A, semantic search, syllabus matching, Ask-the-Book RAG.	OpenRouter API (Gemini 2.5 Flash), SHA-256 AICache

Table 11.2 - Three-tier breakdown

11.6 Module Description

- **Authentication & Access Control Module:** Manages user registration, credential verification, bcrypt password hashing, and JWT session lifecycle. Enforces 4-tier Role-Based Access Control (Student, Faculty, Librarian, Administrator) across all protected frontend routes and backend REST endpoints.
- **Catalog & Inventory Management Module:** Maintains the institutional repository of 2,299 normalized titles and 8,246 physical copies with unique barcodes (e.g. D-1 to D-8246) and physical rack locators (A1–C4). Supports bulk Excel inventory ingestion, multi-attribute filtering, and real-time copy availability tracking.
- **Circulation & Fine Management Module:** Handles automated checkout, return, renewal, and reservation workflows. Automatically computes 14-day due dates, tracks overdue loans, applies programmatic penalties at ₹5/day upon return validation, and supports librarian fee waivers with audit logs.
- **AI Study Assistant & Semantic Search Module:** Integrates Google Gemini 2.5 Flash via OpenRouter for conversational academic Q&A, TF-IDF semantic book search, automated syllabus-to-textbook matching, and PDF study summary generation with SHA-256 response caching.
- **Analytics & Business Intelligence Module:** Aggregates transactional database records into 22 interactive Recharts visualizations, displaying 30-day footfall curves, category holding donuts, top borrowed titles, peak hourly traffic heatmaps, and reading hall seat occupancy.
- **Administrative Governance & Cryptographic Audit Module:** Provides administrators with account provisioning tools, privilege assignment, 99.98% SLA telemetry monitoring, and a tamper-evident SHA-256 hash-chained activity ledger recording every critical system action.

11.7 Database Design

The relational database schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and preserve referential integrity. Deployed on Supabase PostgreSQL, it features foreign key cascading, unique constraints, and B-tree indexes on frequently queried search attributes.

Table Name	Important Columns & Constraints	Functional Purpose
User	id (PK), memberId (UQ), email (UQ), passwordHash, role, department, status	Stores member credentials and 4-tier RBAC authorization flags.
Book	id (PK), title, author, isbn, department, resourceType, embedding	Maintains normalized academic title records and vector embeddings.
BookCopy	id (PK), bookId (FK), barcode (UQ), location, shelfNumber, status	Tracks physical copy inventory mapped to library rack coordinates.
IssueRecord	id (PK), bookCopyId (FK), userId (FK), issuedDate, dueDate, status	Records active loans, return timestamps, and renewal counters.
Fine	id (PK), issueRecordId (FK), userId (FK), amount, daysOverdue, status	Audits overdue financial penalties assessed during book return.
Reservation	id (PK), bookId (FK), userId (FK), queuePosition, status	Manages patron book hold queues with priority scoring.
SeatBooking	id (PK), userId (FK), seatCode, roomType, startTime, endTime, status	Tracks reading hall and digital lab reservations in real time.
ActivityLog	id (PK), userId (FK), action, details, currentHash, previousHash	Provides a tamper-evident SHA-256 cryptographic audit trail.

Table 11.3 - Database tables

```
CREATE TABLE "IssueRecord" (
  "id" TEXT PRIMARY KEY DEFAULT gen_random_uuid(),
  "bookCopyId" TEXT NOT NULL REFERENCES "BookCopy"("id") ON DELETE RESTRICT,
  "userId" TEXT NOT NULL REFERENCES "User"("id") ON DELETE CASCADE,
  "issuedDate" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "dueDate" TIMESTAMP(3) NOT NULL,
  "returnedDate" TIMESTAMP(3),
  "renewCount" INTEGER NOT NULL DEFAULT 0,
  "status" TEXT NOT NULL DEFAULT 'ACTIVE',
  "issuedBy" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Listing 11.1 - IssueRecord table definition

Key Entity Relationships: User (1:N) IssueRecord, Book (1:N) BookCopy, BookCopy (1:N) IssueRecord, IssueRecord (1:N) Fine, User (1:N) Reservation, User (1:N) SeatBooking.

11.8 REST API Endpoints & Integration

The Express backend exposes a comprehensive suite of RESTful endpoints adhering to standard HTTP verbs and JSON serialization. Every endpoint is protected by JWT authentication middleware and validated against declarative Zod schemas.

API Endpoint & Method	Access Authorization	Functional Description
POST /api/auth/login	Public	Authenticates member credentials and issues signed JWT bearer tokens.
GET /api/auth/me	All Roles	Fetches authenticated profile, active loans count, and reading streak metrics.
GET /api/books	Public	Retrieves paginated catalog titles with multi-filter and keyword parameters.
POST /api/books/import-excel	Admin	Parses multipart Excel workbook to bulk insert titles and physical copies.
POST /api/issues/issue	Librarian / Admin	Executes book checkout, verifies eligibility, and assigns 14-day due date.
POST /api/issues/return	Librarian / Admin	Validates book return, updates copy status, and assesses ₹5/day fine if overdue.

POST /api/ai/chat	Authenticated	Processes conversational queries with catalog-aware prompt context.
POST /api/ai/semantic-search	Public	Performs hybrid vector similarity and keyword search across book records.
GET /api/analytics/comprehensive	Admin / Librarian	Generates consolidated data series for 22 Recharts visualization components.

Table 11.4 - Core REST API endpoints

11.9 Security, Auditability & Performance Optimization

- **Authentication & Password Hashing:** User credentials are protected using bcryptjs with 10 salt rounds. Client sessions rely on HMAC-SHA256 signed JWT tokens containing member ID, expiration timestamps, and authorized role claims.
- **Cryptographic Audit Ledger:** Sensitive transactions (book checkouts, catalog imports, fee waivers) append immutable records to the ActivityLog table using SHA-256 hash chaining (currentHash = SHA256(previousHash + userId + action + timestamp)), ensuring complete non-repudiation.
- **Database Connection Pooling:** Backend services utilize PgBouncer on Supabase Cloud to maintain persistent pool connections, eliminating connection starvation during concurrent query bursts.
- **AI Inference Caching:** Prompt-response pairs are cached in the AICache table using SHA-256 prompt hashing with a 24-hour TTL, slashing redundant API calls and maintaining instantaneous sub-100ms response times.

11.10 Project Conclusion

LibraAI successfully achieves its primary objective of modernizing institutional library management through a robust, cloud-native web architecture combined with state-of-the-art generative artificial intelligence. By automating circulation desks, providing real-time Recharts visualization, and integrating conversational study assistants, the system eliminates manual administrative burdens, enforces stringent audit compliance, and elevates the academic learning experience for campus scholars.